From Lean Proofs to Public Claims

Release checks and trust boundaries in one formal mathematics repository

Will Cook

July 2026

Abstract

Lean verifies that a proof establishes the formal statement written in the source. It does not check that a README or paper states that proposition faithfully. This case study separates four questions often compressed into the word "coverage": whether declarations were inventoried, whether they were routed to a mathematical zone, whether selected declarations were interpreted as authored statement nodes, and whether selected public claims were reviewed. At the audited revision every one of 151,755 live declarations has a typed route, but only 4,258 of 141,820 authored theorem-like declarations are linked to authored statement nodes. The first fact is exhaustive routing, not exhaustive interpretation.

For public claims, a reviewed record names Lean evidence, status, finite domain, and the adjacent stronger statement that remains open. Release checks preserve those recorded decisions after edits but do not judge their mathematical correctness. The worked example is exact: Lean has checked a finite certificate at every lcm-diagonal scale $t \leq 82$, whereas the open requirement asks for certificates beyond every cutoff. The development proves that this unbounded supply is equivalent to an affirmative answer to Erdős Problem 249; it does not prove the supply. A historical false edit crossed that boundary without changing Lean and escaped the then-current checker. A repaired negative fixture now rejects that one false neighbour. The perimeter is therefore selective in both directions: it grows only when a person records a relationship, and it shrinks silently when one is retired, since no remaining check demands what is no longer registered. This is evidence for a specific failure and repair, not a detection rate, semantic-truth certificate, or solution of Problems 249 or 257.

1 The publication gap

The repository studied here is a public development in the Lean 4 proof assistant [1] around two unsolved problems in number theory, Erdős Problems 249 and 257. Both problems remain open. The project proves intermediate results, exact reformulations, and finite certificates around them; it does not claim a solution to either problem.

One theorem illustrates the problem this paper is about. Lean has checked a finite certificate at every integer scale $t \leq 82$. A certificate is a finite check proving that a

Authorship and AI use. The prose in this version was generated by large-language-model agents under Will Cook's direction. Cook set the objectives and acceptance criteria, reviewed and authorised the manuscript, and is responsible for its contents. The agents are production tools, not authors. See Appendix A.

particular value is not an integer, although rationality would require it to be one. The corresponding open requirement asks for certificates beyond every fixed cutoff. Whatever the largest checked scale is, a cutoff lies beyond it, and the finite list says nothing there. The finite list therefore does not settle the open problem.

A later README edit can nevertheless overstate the result. By an equivalence proved in the development (Section 3), certificates beyond every fixed cutoff are not merely better evidence. Their existence is equivalent to answering Problem 249 affirmatively: the totient constant is irrational. Three objects recur throughout this paper: the *finite theorem*, certificates at every scale $t \leq 82$; the *open requirement*, certificates beyond every cutoff; and the *equivalence* between the open requirement and the irrationality. Lean has checked the first and the third; no Lean theorem carries the first to the second, and proving that implication would settle the problem. An edit that presents the finite cases as completing the open requirement therefore asserts, in English, exactly the bridge the development does not contain; in substance it announces a solution to an open problem. Every Lean proof remains valid under that edit, because the README is not part of any proof. No program in the repository decides whether a line of English has the same meaning as a formal statement. Lean acceptance therefore does not verify the public description.

The design joins six elements. Its five file-and-workflow parts are Lean source, the maintainer-reviewed claim record, authored public documents, generated indexes and summaries, and the release program and continuous-integration workflow. Authored semantic classifications sit between source and claim record. The repository keeps a *maintainer-reviewed claim record* in `docs/claims.json`. For each selected result the record states the public wording, its status, the named Lean theorems or definitions that support it (Lean calls such named items *declarations*), the bounded domain it covers, and the stronger conclusion the record marks as open. A release checker, `scripts/check_release.py`, then verifies that the recorded relationships still hold across the Lean source, the record itself, the authored public pages, and the generated files. The record covers only *registered claims*, meaning claims entered into it, not every line of prose in the repository; software cannot preserve a relationship it was never pointed at, and Section 5 shows this limit operating in practice. The shortest accurate summary of the division of labour is:

> Lean checks the formal proofs. A maintainer reviews what those proofs mean and how they may be described. The release machinery checks that the recorded relationships remain intact before a release.

Readers focused on the release design may skip Sections 3.1–3.2.

2 The release workflow

Read the upper half of Figure 1 from left to right: Lean source, semantic and claim records, and public documents, with generated views built from those records. The dashed arrows show human review. The lower half is automated: each job runs on its own fresh copy. The decision is: do both jobs pass?

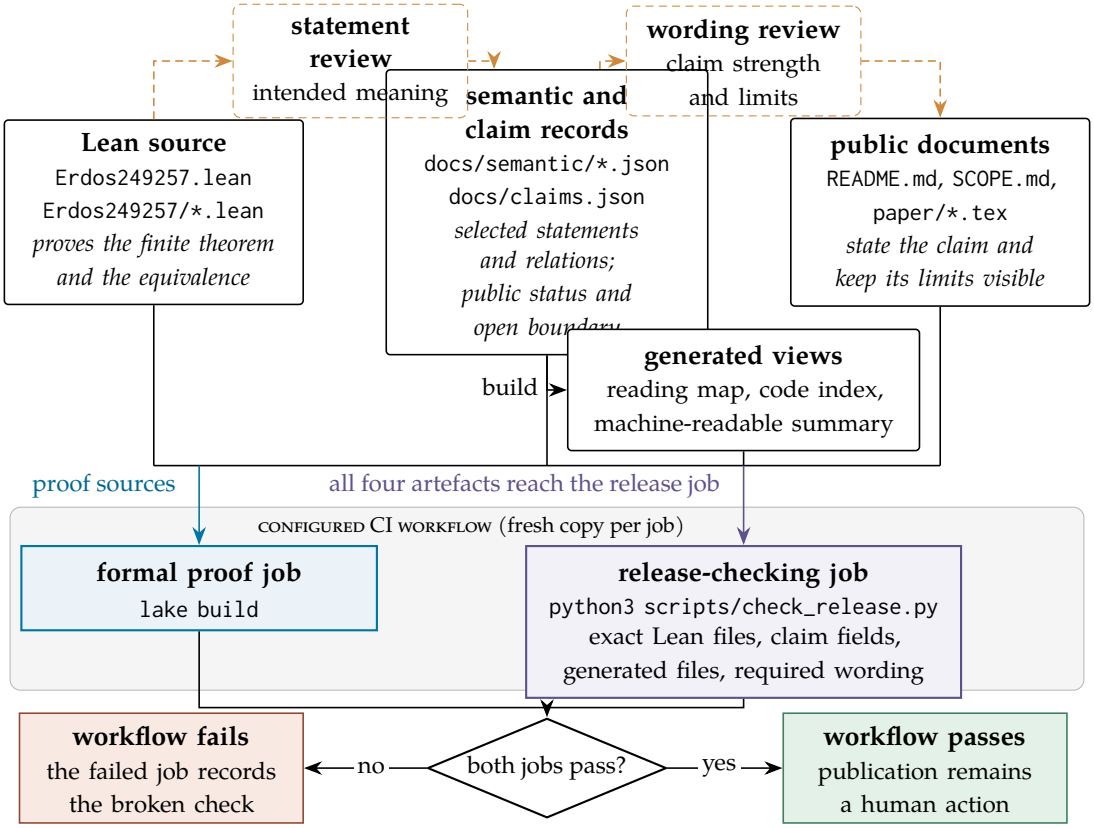

Figure 1: The configured checks for one saved change. Dashed elements are human: one maintainer performed both review steps here. Solid elements are files, programs, and automated checks. Lean checks the formal proofs. Human review compares their intended meaning with authored classifications and the recorded public wording and limits. The release job checks that recorded names, wording, Lean files, and generated views still agree. Continuous integration reruns the Lean and release jobs; neither interprets unrestricted prose. In the worked example of Section 3, the release job protects the recorded distinction between the finite theorem and the open requirement. Blue marks the Lean job, amber review, and violet the release job.

Someone must compare a formal theorem with its public wording and judge whether the wording is faithful within stated limits. Once that judgement is recorded, a program can test whether later changes satisfy the recorded rules about names, files, and wording. It does not decide whether the original judgement was correct.

The workflow does not technically force a second independent mathematician. A single maintainer can edit a Lean statement, its claim record, and the public wording together, and every check can pass while the shared interpretation is wrong; Section 6 applies this limit to the worked claim. The checks detect departures from reviewed decisions. They do not replace review or add authority of their own.

The two jobs answer different questions. `lake build` asks whether Lean accepts the imported formal statements and proofs. It knows nothing about the README,

and it would still accept a change whose README falsely states that the open requirement has been met, because no public document is among its inputs. The second job, `python3 scripts/check_release.py`, asks whether the recorded relationships among proofs, record, public pages, and generated files still hold; it checks that the Lean source matches the recorded version but does not run Lean. The workflow in `.github/workflows/lean.yml` runs them as two separate jobs whenever a saved change is sent to GitHub, either directly or for review.

Each job runs on a new GitHub-hosted virtual machine (a *runner*) and downloads the selected repository version (a *revision*) [2]. A checker error stops the program; the workflow treats that exit as a failing job. Repository settings decide whether that blocks release. Failure can stop the configured release process; a passing release job proves no mathematics.

The generated views reorganise the records into a reading map, code index, and machine-readable summaries. They create no new mathematics. Each view names its builder, and stale output is regenerated rather than edited. The complete file map, ownership table, and maintenance commands live in `ARCHITECTURE.md`; this paper gives only what this argument needs.

3 One claim from Lean theorem to public page

We now follow one result from Lean source to public page. We begin with its public meaning, not its internal name:

> Lean has checked a finite certificate at every lcm-diagonal scale $t \leq 82$. The registered open requirement asks for certificates beyond every fixed cutoff. This finite interval does not settle it.

3.1 What the certificates are for

Erdős Problem 249 asks whether the totient constant

$$S = \sum_{n \geq 1} \frac{\varphi(n)}{2^n}$$

is *irrational* [3], meaning that it is not a/b for any integers a and b with $b \neq 0$. Here φ is Euler's totient function: $\varphi(n)$ counts the integers from 1 to n sharing no factor greater than 1 with n. The following exact equivalence explains why the certificates matter. Multiplying S by 2^N makes its first N terms integral; write R_N for the remaining rescaled tail. Thus R_N differs from $2^N S$ by an integer. Put $D_{N,h} = R_{N+h} - R_N$. It differs from $2^N(2^h - 1)S$ by an integer. For $h > 0$, the factor $2^N(2^h - 1)$ is a nonzero integer, so an integral $D_{N,h}$ would force S to be rational. Conversely, every rational number has an eventually repeating binary expansion, the base-2 analogue of a repeating decimal. Thus some $h > 0$ makes $D_{N,h}$ integral for every sufficiently large N. Lean checks the exact equivalence:

$$S \text{ is irrational} \quad \Longleftrightarrow \quad D_{N,h} \notin \mathbb{Z} \text{ for every } h > 0 \text{ and every } N.$$

For fixed h and N, Lean proves $D_{N,h} \notin \mathbb{Z}$ exactly when a certificate exists at a finite depth L. Its integer calculation approximates $2^L D_{N,h}$ with error at most $r = N + h + L + 2$. The certificate checks that the residue modulo 2^L lies strictly between r and $2^L - r$, outside both bands compatible with an integral value. The omitted infinite tail is thus accounted for by an explicit bound. The claim record calls this a *certified kill*. We use the shorter word *certificate*: it proves only that this particular tail difference is not an integer.

The scales of the finite theorem compress this two-parameter requirement to one. The diagonal $H_t = \operatorname{lcm}(1, \dots, t)$ is the least common multiple of $1, \dots, t$: the smallest positive integer divisible by every integer in that list. It is therefore divisible by every candidate period up to t. If S were rational, its binary digits would have a period h_0 after a position N_0. For $t \geq \max(h_0, N_0)$, one has $h_0 \mid H_t$ and $H_t \geq N_0$, so rationality would make D_{H_t,H_t} integral; a diagonal certificate rules this out. The development proves the reduction exact: certificates along the diagonal beyond every fixed cutoff are equivalent to the full two-parameter requirement, and therefore, through the chain above, to the irrationality itself. Certificates on a finite list, however long, are not.

Writing $\mathrm{Cert}(t)$ for "a certificate exists at scale t", the two statements then read:

$$\begin{aligned} \text{checked:}\ & \mathrm{Cert}(t) \text{ for every integer } t \leq 82; \\ \text{open:}\ & \text{for every cutoff } T,\ \mathrm{Cert}(t) \text{ for some } t > T. \end{aligned}$$

The boundary between them is a matter of logical form, not of scale. The bound 82, 820, or any other fixed bound stands in the same relation to the open requirement: a larger cutoff still exists. No finite interval implies the second statement without a theorem that produces new witnesses. In particular, no certificate at $t = 83$, and no cofinal supply, is claimed. This is the boundary an edit can silently delete.

3.2 The formal evidence

The historical module `Erdos249257/DiagonalPincerCertificatesT64.lean` deposits certificates at 28 prime-power breakpoints through $t = 64$. Because H_t is constant between breakpoints, its combined theorem already covers every $t \leq 66$. The later module `ErdosProblems/Skip/LadderT67.lean` adds the remaining finite arithmetic and proves `exists_diagonalKill_le_82`: for every $t \leq 82$, there is a checking depth and a certificate at period and position H_t. The claim record names six supporting declarations across the two Lean libraries.

Lean's *kernel* is the small trusted program that checks every accepted proof. These finite certificate proofs use `decide`, which evaluates a finite proposition and supplies a proof term for the kernel [4]. Separately, `Erdos249257/LcmConeFlatness.lean` proves the exact tail-difference, pointwise-certificate, and diagonal equivalences, ending at `irrational_totient_series_iff_lcm_diagonal_certificate_supply`. The default `lake build` compiles the libraries rooted at `Erdos249257.lean` and `ErdosProblems.lean`, including the module that proves the $t \leq 82$ theorem.

3.3 Four meanings of coverage

Coverage is a vector, not one percentage. First, the declaration atlas inventories every declaration in the pinned source. Second, every live declaration receives one role-and-

zone route, or belongs to a generated family. Zero routing orphans means every declaration can be found; it does not mean every theorem has been interpreted. Third, selected declarations are linked to authored statement nodes, and selected nodes have typed mathematical relations with stated bases. Fourth, the claim registry selects a much smaller set for public presentation and records its status and open boundary.

The generated receipt at this revision makes the ceiling visible: 151,755 of 151,755 declarations are routed, with no duplicate routing receipts; 4,258 of 141,820 authored theorem-like declarations are linked to authored statement nodes, while 137,562 are only zone-routed; and all 299 declarations selected by the public claim ledger have the required semantic route. The command `python3 scripts/query_semantic.py coverage` derives these counts. Its checker validates source freshness, roles, node references, relation labels, endpoints, and stated bases. It does not prove that canonical prose is extensionally equivalent to its Lean proposition, that a mathematical relation is true, or that the selected graph is complete. Here *semantic coverage* means explicit statement-level interpretation, never mere inventory or routing.

3.4 From a cold clone to a proof receipt

The public checkout commits three different navigation products. The declaration atlas inventories 151,755 declarations in 991 Lean modules. The semantic graph adds authored meaning to a selective subset, as the preceding counts show. The elaborated dependency index records 8,772 source-resolved nodes and 38,844 direct value-reference edges for the two loaded library roots. It also reports its misses: 3,074 atlas declarations are not resolved to a node in that loaded environment, and 5,800 public reference pairs have an external or otherwise unresolved endpoint. The first command, `python3 scripts/query_corpus.py --tour --format card`, derives a five-line orientation from those committed products: corpus scale, loaded-root graph size and unresolved counts, the authority boundary, the exact open frontier, and the available intent classes. It does not name a specimen module in its routing logic. The longer `agent_native_corpus_navigation` route then gives generic declaration inventory, connection-card, proof-cone, workbench, and focused-build commands. Both surfaces work immediately in a cold clone and do not elaborate Lean. That is a navigation guarantee, not a proof guarantee.

Proof work crosses a deliberate authority boundary. The session notary in `scripts/proof_workbench.py` records observations, falsifiable conjectures, plans, interpretations, abandonments, probes, and claims in an append-only ledger. A probe stores the exact Lean input bytes and computes its verdict from the pinned Lean process; an agent cannot author that verdict. A claim must cite a kernel-accepted probe receipt or the notary refuses it. Replaying a session reruns every stored probe and compares the new verdict with the recorded one. Static inventory and dependency queries nominate where to look; only the probe can establish that the proposed Lean source is accepted.

The committed prospective carry-pivot session is a dogfood case rather than a synthetic demonstration. Its ledger contains six reasoning notes, one kernel-accepted probe, and two claims bound to that probe. The session produced `Erdos249257/SuffixCylinderCarryPivot.lean`, including an exact divisor-incidence

identity that sharpens a previously landed inequality to an equality. At this revision `proof_workbench.py replay --session carry_pivot_2026_07_27` replays the stored probe with a matching `kernel_accepted` verdict. The receipt establishes acceptance and replay integrity. It does not establish that the reasoning was optimal, that a bounded search was exhaustive, or that the mathematical result is new outside the repository history.

Compilation is separate again. A fresh Git clone contains the navigation products but not platform-specific project `.olean` files. Formal editing therefore needs a first project build or a compatible restored cache. Thereafter `scripts/lean_fast_build.py` can select an exact target, modules changed since a Git reference, or the stale dependency cone. Lake content traces are used for restored caches so checkout timestamps do not force a full rebuild. In the dogfood environment a two-root trace plan reported zero stale modules; replaying the historical four-file problem-owned addition selected 26 modules in four dependency waves rather than both complete roots. The 24-test planner suite checks trace freshness, changed-file selection, dependency ordering, bounded parallelism, failure handling, and the final serialized Lake authority check. These are one-revision functional receipts, not cross-machine performance benchmarks.

The exhaustive dependency-index validator also writes an exact receipt under the ignored `.lake` cache after a full environment export matches the committed index. It binds every supported Lean source file, the pinned toolchain and Lake locks, the exporter and builder inputs, the declaration atlas and generated manifest, the formal-source release slice of the claim registry, the dependency-extraction helper definitions, and the index bytes. An ordinary check reuses the receipt only while all of those bytes are unchanged; an explicit full-check flag bypasses it. Thus restored CI caches avoid repeating the environment export for documentation-only revisions without allowing a changed formal input to inherit an old validation result.

3.5 The reviewed record

The quoted record entry contains four names specific to this repository. A *certified kill* is the certificate defined in Section 3.1. The entry abbreviates least common multiple as "lcm", so its *lcm-diagonal scales* are the values H_t. The *small periods* are the periods 1 through 8, each handled by one explicit certificate. A *certificate shard* is the same calculation at a fixed period, position, and depth, isolated in another Lean file and checked by `decide`.

Table 1 shows the substance of the corresponding entry in `docs/claims.json`, ordered as a reader meets it: what is claimed, where it stops, what remains open, then the supporting machinery. The entry also records an exact source line for each declaration; those coordinates are omitted from the table.

Field	Content
Public statement	"Lean checks every lcm-diagonal scale $t \leq 82$ and the listed finite shards. The historical 28 deposits through $t = 64$ already covered $t \leq 66$ because the lcm is constant between prime powers."
Bounded domain	Listed small periods, every $t \leq 82$, and the fixed parameters of each finite shard; no $t = 83$ or cofinal claim.
Remaining open	A link to the registered open requirement *unbounded certificate supply*: produce a certified witness beyond every fixed cutoff.
Status	*verified finite instance*: Lean checked the stated finite inputs.
Supporting declarations	Six named Lean theorems with recorded modules and source lines, ending in `exists_diagonalKill_le_82`.
Claim identifier	`certified_kill_instances`

Table 1: The reviewed claim entry for the worked example, exact source coordinates omitted. Together, the positive statement, bounded domain, and remaining-open link distinguish the finite result from the open problem. In the failure of Section 5, overstatement came from deleting this boundary, not from inventing a theorem.

The table records the claim rather than explaining it. The remaining-open field points to the open requirement of Section 3.1, while the status *verified finite instance* classifies the finite theorem. Under the equivalence the remaining-open field is part of the claim: without it, the entry no longer separates the finite theorem from an answer to Problem 249. The record gives explanatory prose, the release checker, and generated views one stable reference. Prose must still bridge the gap between an exact claim and a reader's understanding.

The fields differ in kind, and the difference matters for what "reviewed" means. The public statement, bounded domain, remaining-open link, and status are fields about meaning. They record a human judgement, and only a person can meaningfully change them. The record fixes the allowed status terms, and the checker rejects any other status. The claim identifier lets software find the claim; it appears last, not in place of the wording, and the per-declaration source lines are location data that a program may refresh when code moves; updating them does not repeat the judgement about meaning. No field can by itself establish that the English is faithful to the Lean: the maintainer judged that the public statement above describes the six declarations correctly, and the record stores the outcome of that judgement, not a justification of it. Review and authorisation are distinct: review asks whether the wording is faithful; authorisation decides which wording the project will publish. One maintainer performed both here. The review itself is concrete: read the final theorem, check that the wording claims the listed scales and no more, confirm that the open requirement stays named as open. It is also an event, not a permanent property: the review covered particular declarations and wording. A later change to either reopens it; later checks can only preserve what was recorded. That decision must survive ordinary rewrites months later.

3.6 What the checker verifies here

For this claim, `scripts/check_release.py` verifies a finite, listed set of relationships. Each declaration name must appear in the Lean source within three lines of its recorded coordinate. This checks that the name remains near its recorded position, not that the declarations entail the public wording. The checked Lean source itself must match the saved Git revision named in the claim record. The checker compares the top-level Lean file and the entire library directory byte for byte with that recorded version. It rejects any difference or extra Lean file absent from that version, so the verified names cannot refer to an older source than the one being shipped. The required limitation wording must remain present on the registered public pages; for this claim that wording is the visible trace of the boundary of Section 3.1, the clause keeping the open requirement open. The generated views must equal a fresh rebuild from their sources. Presence, identity, and proximity are all the checker can test. Lean checks the theorem; the checker checks the recorded description around it; a person remains responsible for the claim that the two say the same thing.

4 What the checks establish

The word "verification" is easy to overread here. Four separate questions are involved. Did Lean accept the formal statements and proofs? Did a maintainer record a judgement that selected public wording describes them within stated limits? Do the configured structural comparisons pass at this revision? Did continuous integration run the configured jobs on that revision and report success? Each question needs different evidence. A yes to one does not settle the others.

When both jobs pass, Lean has accepted the proofs and each configured structural comparison has passed. This does not repeat the maintainer's review, establish that the review was correct, or show that every consequential claim was registered. Table 2 states what each layer can and cannot establish. Keeping the layers separate prevents the single word "verified" from making their combined guarantee sound stronger than it is.

Instantiated on the worked example, the source proves the finite theorem and equivalence, not the open requirement. Maintainer-reviewed wording confines the theorem to the listed scales. The release checker preserves declaration coordinates, revision agreement, the limitation clause, and rebuilt views; continuous integration reruns the jobs on fresh copies.

The limits follow the same lines. Lean cannot notice English that quietly implies the open requirement met. The recorded review does not show that every page repeating the claim was found. The checker cannot reject a paraphrase it was never given. Passing both jobs cannot make a mistaken reading of the equivalence correct.

The release checker rejects proof placeholders, project-defined axioms, and native evaluation; ordinary `decide` produces a kernel-checked proof term [4, 5]. It compares reviewed sources, rebuilds views, verifies paper hashes, and reruns selected negative fixtures. These checks preserve recorded relationships, not mathematical meaning or unseen faults.

Layer	What it can establish	What it cannot establish
Lean build and kernel	The written formal statements are proved from their imported definitions and assumptions, using the recorded Lean version and dependencies.	That a formal statement is the one the author intended, or that any English description of it is faithful.
Maintainer review	A recorded judgement that a named formal statement has the intended meaning and selected public wording describes it within stated limits.	That Lean accepts the proofs; that every important claim was registered; or that the review was independent or correct.
Release checker	That recorded names, source lines, fields, links, required wording, Lean files, and generated files satisfy the declared rules; that prohibited proof shortcuts are absent; and that deliberately wrong examples still fail.	What unregistered prose means; whether the record is complete; or whether the human judgements it preserves are correct.
Continuous integration	That each job ran on its own fresh copy of the uploaded revision and exited successfully.	Independent mathematical approval; anything beyond what the two jobs themselves establish.

Table 2: What each layer establishes and leaves open.

For a mathematical change, run `lake build`; review statements, assumptions, and meaning; regenerate views; then run `python3 scripts/check_release.py`. Continuous integration reruns the two separate jobs. Prose-only edits still need human and release review; stronger wording reopens the judgement. The repository guide lists the full commands.

5 A boundary the checklist missed

The evidence comes from three different times. The historical exercise, the present post-repair test, and the later executable reconstruction answer different questions and must not be merged. Appendix A identifies their files and commands.

Historical exercise. The structured report in `docs/publication_evidence.json` identifies a saved Git revision. It says that ten deliberate false edits were applied to a separate copy one at a time and ran the release checker, but not Lean. They covered seven configured relationship kinds, including status, source coordinates, boundary wording, generated-file freshness, and size budgets. The original run logs were not retained; the file is a report, not raw output.

According to that record, nine of the ten edits were rejected. One escaped: the README clause saying that the finite cases *do not supply* the open requirement was changed to say that they *complete* it, asserting exactly the missing bridge to Problem 249.

Lean was untouched, and the checker passed because the clause was absent from its checklist. The escape shows that a passing checker does not certify all public prose; it gives no detection rate.

Repair and present witness. The repair required the clause and added a *boundary witness*: a false neighbour crossing a consequential public boundary while leaving every Lean proof intact. The post-repair witness accepts the current README and rejects a test copy containing the false clause, distinguishing that neighbour. In the release check's exact terms, the current README passes and a false copy, the boundary witness, fails. The other nine edits were not rerun against the extended checklist, so there is no post-repair aggregate result.

Later reconstruction. The executable reconstruction preserves three original targets and uses seven documented replacements. It is a new experiment, not the missing runs.

The evidence marks a coverage boundary, not a reliability score. The edits were authored by the checker's author; all seven relationship kinds were represented, but five of them by one edit each; and no controlled comparison with disciplined manual review was run. Reader error, ordinary use, and transfer were not measured.
The retained evidence supports one design lesson:

> The program can preserve a relationship only after a person has identified and recorded that relationship.

Requiring a clause to be present does not detect a contradictory stronger claim elsewhere on the page. Coverage grows only when a person notices and records a relationship; selected high-risk commitments also have a negative fixture. Coverage can also shrink: deleting a registered relationship leaves no rule requiring it, so every remaining check passes. Retiring a commitment therefore deserves the same review as creating one.

The shape of this limit is familiar from Section 3.1. A configured checklist, like a finite certificate interval, does not exhaust an open-ended domain. The comparison is only structural: prose has no equivalence theorem and no diagonal compresses it.

Three boundaries the companion papers exposed. The exercise above was designed against the checklist. Three corrections made to this project's own mathematics papers over the same period were not, and each sits outside the perimeter for a different reason. A headline theorem in the Problem 249 note is kernel-checked but owned by no record entry, so it carries no reviewed public status while reading as a result; the checker is silent because the record defines the perimeter, and absence from it is not a detectable state. A support the Problem 257 note printed as open had in fact been settled in the literature in 2019; no edit, no proof and no comparison could have signalled it, since the defeating evidence lay outside the repository. And a valid parity theorem in that note was read as a frontier of the value until it was observed that adjoining one element to the support changes the value by a rational number and removes the obstruction — the theorem was correct and its advertised significance was not. Registration gap, stale external status, and representation-dependent significance are three distinct classes, none

reachable by comparing recorded artefacts with one another. Naturalistic rather than injected, they say more about where the perimeter ends than the designed exercise; like it, they are incidents, not a rate.

6 Scope, reuse, and limits

The failures the design addresses. The checks address accidental disagreement after a sound review. A declaration may move while its recorded line number stays fixed, and a generated view may be stale or hand-edited. A rewrite may remove a required limitation, a status may be upgraded, or the shipped Lean files may cease to be the reviewed ones. A check may also decay until it can no longer fail. In each case, one recorded item disagrees with the others, which a mechanical comparison can detect.

The failures it does not address. Three failure modes remain outside the design. *Unregistered wording:* a paraphrase can strengthen a claim without touching a registered anchor, and a contradiction, misleading emphasis, or new public document can escape for the same reason. Requiring one clause to be present does not require the page to agree with it. *Coordinated but wrong change:* a maintainer can alter source, record, and prose together, so every comparison agrees. If the certificate definition were weakened while the record and prose were updated to match, all checks could pass although the public reading of the equivalence was wrong. *Mistaken review:* if the original judgement was wrong, the machinery preserves the mistake. It checks persistence, not the quality of the judgement.

Limits of the design and this implementation. Some limits follow from the design. Claims remain in unrestricted prose beside an external record, so human interpretation cannot be eliminated. Only listed relationships are checked. The repository calls this set its *assurance perimeter*: recorded commitments inside it trigger automatic checks; outside it, the checker is silent. Choosing what to record remains a human judgement. This repository gives priority to headline results and to wording whose accidental strengthening would change the project's public status, as the deleted boundary clause did under the equivalence. If the same claim appears on several pages, the checker sees only the appearances named in the record. Other limits are implementation choices: one maintainer performs both review and authorisation; the checker accepts a theorem name within three lines of its recorded location; anchors require exact wording; the exercise used one edit per relationship type; and the original logs were not retained.

Reuse. Another formal project could reuse the pattern without copying any file format. It applies when bounded formal evidence sits beside a stronger public target. Examples are finite cases beside a universal statement; a conditional theorem beside its unresolved hypothesis; one direction of an implication beside a claimed equivalence; or a result under named assumptions beside an unconditional headline. What transfers is the reviewed boundary between what is established and the adjacent stronger statement that is not. The minimum obligations are concrete. Each public claim needs its own record entry, named formal evidence, and the exact source version containing it. Its scope, assumptions, and the stronger conclusions it does not establish must be explicit. Human

review is distinct from automated checking. Each generated public view needs a named builder. Proof checking and publication checking must be separate and able to fail. Every important recorded relationship must state its mechanical check; selected high-risk boundaries should also have a deliberately false fixture. The record must present its coverage as selective rather than complete.

What is not established. The architecture does not establish a solution to either Erdős problem; that a formal statement is the statement the author intended; that software understood any unregistered prose; that the record is complete; that one maintainer's review is independent or adequate; or that the design transfers to other projects with its behaviour intact. A large number of passing comparisons measure none of those things. What a passing workflow establishes is narrower: the configured jobs ran on the named revision, Lean accepted the formal proofs, and every configured structural comparison passed. This paper is an architecture note with one bounded case study and three naturalistic incidents, not an empirical evaluation of the design. A credible evaluation would need review records naming reviewer and revision, wrong edits authored by someone other than the checker's author, and only then a comparison with ordinary review on the same changes. None of those stages exists here, and this paper claims none of them.

7 Related systems

A *proof blueprint* pairs an informal outline with Lean declarations and records which results depend on which earlier results. `leanblueprint` stores that outline in TEX and checks that every named declaration exists [6]. LeanArchitect infers dependencies and `sorry` status, exporting synced TEX [7]. Text and unformalised nodes stay manual. The Carleson blueprint documents the same relationship at project scale: an early blueprint initiated the Lean work, roughly 180 tasks were distributed, and formalisation feedback corrected and extended the final guide [8]. These systems support a formalisation in progress. The claim record instead asks which public wording was reviewed after Lean accepted a proof, and which stronger conclusion remains open.

LeanDojo supplies an open programmatic Lean environment, extracted proof data, premise annotations, retrieval, and a 98,734-theorem benchmark [9]. Pantograph supplies Python, REPL, and FFI interfaces for tactic execution, proof-state exploration, metavariable coupling, and data extraction [10]. They address the machine-to-Lean interaction boundary. The workbench here is complementary and smaller: it does not provide a learned search policy or replace those interfaces; it notarises the policy moves an agent chose, prevents an authored kernel verdict, and binds claims to replayable accepted probes.

Large-library maintenance is itself prior art, not an incidental implementation concern. *Growing Mathlib* describes deprecation, semantic linters, conscious library redesign for compilation speed, explicit technical debt tracking, per-commit benchmarks, and review/triage tooling [11]. The focused and trace-based build wrapper used here applies that maintenance posture to a much smaller two-root corpus. Its local receipts do not transfer Mathlib's scale, performance, or social evidence.

Lean Atlas and EconCSLib ask whether a formalisation expresses its intended source mathematics. Lean Atlas leaves semantic verification to people. Given chosen theorem statements, its Lean Compass selects the project declarations whose meaning can affect them, narrowing what a person must inspect; it assumes Lean's standard library and the mathematical library Mathlib are semantically correct rather than inspecting them [12]. This guarantee is conditional on choosing all targets that matter; it does not find every claim occurrence in later public prose. EconCSLib instead has a model write Lean; Lean checks statements and proofs; two model passes and a dashboard assess source-to-Lean translation. The model validations currently run in one Codex agent stack, so errors may correlate; dashboard-based human review was sparse and incomplete [13]. This paper addresses a later boundary: it starts from accepted Lean proofs and asks how subsequent public wording can remain within a reviewed interpretation of them.

A recent audit of formal-theorem benchmarks likewise shows that kernel acceptance does not establish fidelity to the intended natural-language problem. Static checks span 13 variants; prompts tested on 92 labelled problems and need human filtering [14]. That work concerns incoming benchmark statements. The present repository concerns outgoing, post-proof claims that recur and change across public surfaces. Its current mechanical contribution is narrower than semantic verification: it makes exhaustive routing, selective statement interpretation, and selected public review separately countable. It still lacks a closed-world census proving that every claim-bearing passage in every public surface was selected.

Isabelle/DOF, a document system built on the Isabelle proof assistant, places formal and informal material in one checked document. Authors define an *ontology*: document classes with typed fields and rules. They label passages with those classes, and Isabelle's editor reports rule violations as they edit [15]. The bounded domain and open conclusion in Table 1 could be typed fields in such an ontology. This repository keeps prose unrestricted and checks a separate record; its checker cannot inspect prose that the record does not name.

CASCADE derives tests and a second implementation from the same documentation using language models. It reports a likely inconsistency only when a generated test fails on existing code but passes on code generated from that documentation. A person must confirm it [16]. Unlike this checker, it can inspect unrecorded documentation.

Mutation testing asks whether tests detect deliberately introduced faults [17, 18]. The deliberately wrong copies in Section 5 serve that narrower purpose: a known wrong input must continue to fail. The ten-edit exercise is a small, hand-authored mutation run. Since nine edits were not rerun after repair, it gives no post-repair detection rate.

Requirements traceability follows a requirement from its origin through development, deployment, use, and revision [19]. An assurance case sets out a system claim, the argument for it, and supporting evidence. The Goal Structuring Notation (GSN) is a graphical notation for that argument structure [20]. The claim record resembles both, but it does not contain an argument that its Lean declarations justify the public wording. It records the approved wording, supporting declarations, scope, and limits; the justification remains a human judgement.

8 Conclusion

Lean has checked certificates at every lcm-diagonal scale $t \leq 82$. It has not checked a certificate at $t = 83$ or an unbounded supply, and the development proves that the latter is equivalent to Problem 249. Problems 249 and 257 remain open.

The systems contribution is an executable distinction: declaration inventory and routing can be exhaustive while statement-level interpretation and public claim review remain selective. The release gate preserves configured, reviewed boundaries and a negative fixture now catches one historically escaped strengthening. The governing rule the failure yields is that an assurance perimeter is only ever as wide as the relationships someone chose to record, so retiring one is as consequential as adding one and deserves the same review. None of this establishes semantic truth, complete selection of public claims, a detection rate, or transfer to another repository. A passing workflow says only that Lean accepted the formal proofs and the configured structural checks passed on the named revision.

A Reproducibility

The reviewed claim record, `docs/claims.json`, names the saved Git revision of the Lean source, which the release checker requires to match exactly. This paper omits the changing commit identifier so that the claim record is the only place that states it.

Declaration of generative AI use. Large-language-model agents were used throughout development to draft and revise prose, formal proofs, and software. The author set the objectives and acceptance criteria, selected and reviewed the claims, and approved the published version. The author assumes responsibility for the accuracy, interpretation, and presentation of the work. Generative systems are production tools; they are not authors and supply no independent authority. That boundary is the subject of this paper as well as a condition of it: the checker described in Section 4 tests recorded relationships a person selected, so passing it does not make generated wording faithful. The author selected, reviewed, and authorised the public claim-to-declaration mappings, and remains responsible for connecting the proofs to public wording.

The paper inventory, `docs/publication_contract.json`, records source and PDF cryptographic hashes and validation commands. The evidence file for Section 5, `docs/publication_evidence.json`, records the protocol, outcomes, and limitations. The reconstruction file, `experiments/publication_mutations.json`, specifies the ten edits. The script `scripts/run_publication_mutations.py` checks that each edit applies once (`--verify-operators`) and, by default, runs all ten in a copy of the saved evaluation version (`--all`). For an edit whose exact original target was not preserved, the file names a fixed replacement. The original raw outputs were not retained; the reconstruction does not claim to be them.

The papers build with Tectonic or standard LaTeX via `make -C paper`. The tracked root PDF is the shipped copy, and the inventory checks its file hash. These identities name artefacts; they do not interpret them.

References

[1] L. de Moura and S. Ullrich, *The Lean 4 Theorem Prover and Programming Language*, in *Automated Deduction—CADE 28*, Lecture Notes in Computer Science 12699, 2021, pp. 625–635, DOI.

[2] GitHub, *GitHub-hosted runners*, documentation, accessed 18 July 2026.

[3] P. Erdős and R. L. Graham, *Old and New Problems and Results in Combinatorial Number Theory*, Monographies de L'Enseignement Mathématique 28, L'Enseignement Mathématique, Université de Genève, 1980, p. 61, scan.

[4] Lean project, *The Lean Language Reference: Validating a Lean Proof*, documentation, accessed 18 July 2026.

[5] Lean project, *The Lean Language Reference: Axioms*, documentation, accessed 18 July 2026.

[6] P. Massot, *leanblueprint*, plasTeX plugin for Lean formalisation blueprints, 2020, software repository, accessed 18 July 2026.

[7] T. Zhu, P. Monticone, S. Welleck, and J. Avigad, *LeanArchitect: Automating Blueprint Generation for Humans and AI*, in *17th International Conference on Interactive Theorem Proving*, LIPIcs 382, 2026, pp. 25:1–25:16, DOI.

[8] L. Becker et al., *A Blueprint for the Formalization of Carleson's Theorem on Convergence of Fourier Series*, 2025, arXiv:2405.06423.

[9] K. Yang, A. M. Swope, A. Gu, R. Chalamala, P. Song, S. Yu, S. Godil, R. Prenger, and A. Anandkumar, *LeanDojo: Theorem Proving with Retrieval-Augmented Language Models*, in *Advances in Neural Information Processing Systems 36*, 2023, arXiv:2306.15626.

[10] L. Aniva, C. Sun, B. Miranda, C. Barrett, and S. Koyejo, *Pantograph: A Machine-to-Machine Interaction Interface for Advanced Theorem Proving, High Level Reasoning, and Data Extraction in Lean 4*, in *Tools and Algorithms for the Construction and Analysis of Systems*, 2025, pp. 116–137, DOI.

[11] A. Baanen, M. R. Ballard, J. Commelin, B. Ginge Chen, M. Rothgang, and D. Testa, *Growing Mathlib: Maintenance of a Large Scale Mathematical Library*, in *Intelligent Computer Mathematics*, 2025, arXiv:2508.21593.

[12] B. Yanahama and A. Sannai, *Lean Atlas: An Integrated Proof Environment for Scalable Human–AI Collaborative Formalization*, 2026, arXiv:2604.16347.

[13] N. Garg, *EconCSLib: AI-Assisted Lean Formalization for Economics & Computation research*, 2026, arXiv:2606.13306.

[14] P. S. Ammanamanchi, S. Bhat, and S. Biderman, *Faults in Our Formal Benchmarking: Dataset Defects and Evaluation Failures in Lean Theorem Proving*, 2026, arXiv:2606.29493.

[15] A. D. Brucker and B. Wolff, *Isabelle/DOF: Design and Implementation*, in *Software Engineering and Formal Methods*, Lecture Notes in Computer Science 11724, 2019, pp. 275–292, DOI.

[16] T. Kiecker, J. A. Sparka, M. Reuter, A. Ziegler, and L. Grunske, *CASCADE: Detecting Inconsistencies between Code and Documentation with Automatic Test Generation*, *Proceedings of the ACM on Software Engineering* 3 (FSE), Article FSE168, July 2026, pp. 3816–3838, DOI.

[17] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, *Hints on test data selection: Help for the practicing programmer*, IEEE Computer 11(4), 1978, pp. 34–41, DOI.

[18] Y. Jia and M. Harman, *An analysis and survey of the development of mutation testing*, IEEE Transactions on Software Engineering 37(5), 2011, pp. 649–678, DOI.

[19] O. C. Z. Gotel and A. C. W. Finkelstein, *An analysis of the requirements traceability problem*, in *Proc. First IEEE International Conference on Requirements Engineering*, 1994, pp. 94–101, DOI.

[20] Assurance Case Working Group, *Goal Structuring Notation Community Standard (Version 3)*, Safety-Critical Systems Club, 4 May 2021, standard.